

实验报告：大模型定点知识编辑与评估

实验方向：03-大模型知识编辑 (Knowledge Editing for LLMs)

1 实验背景与目的

随着大语言模型（LLM）规模的不断扩大，如何高效、精准地更新其内部存储的过时知识或修正错误事实成为一项重要挑战。传统的全量微调（Fine-tuning）不仅耗时耗力，且极易导致模型出现“灾难性遗忘”。

本实验旨在通过实践主流的知识编辑算法——**ROME (Rank-One Model Editing)** 与 **MEMIT (Mass-Editing Memory in Transformer)**，深入理解模型事实知识的存储机制（即 FFN 层作为 Key-Value 存储的原理）。实验重点考察知识编辑的三个维度：**可靠性 (ES)**、**泛化性 (PS)** 和 **局部性 (NS)**。

2 Task 1: 基线测试分析

在进行任何编辑之前，首先通过 `baseline.py` 测试原始模型 GPT-2 XL 对 10 条事实的预置认知。

实验观察

根据图 2-1 基线测试，模型在编辑前完全受限于其训练阶段的原始认知：

```
(my_env) root@autodl-container-654f118e52-2f9b91b5:~/autodl-tmp/my
预期目标 (编辑后): CO2
原始 Ground Truth: H2O
模型当前输出: H 2 O.

The chemical symbol for
-----
[8/10] Prompt: The first president of the United States was
预期目标 (编辑后): Abraham Lincoln
原始 Ground Truth: George Washington
模型当前输出: a slave owner. The first president of the United
-----
[9/10] Prompt: The main ingredient in guacamole is
预期目标 (编辑后): Tomato
原始 Ground Truth: Avocado
模型当前输出: the avocado, which is a fruit. The avocado
-----
[10/10] Prompt: The planet closest to the Sun is
预期目标 (编辑后): Venus
原始 Ground Truth: Mercury
模型当前输出: called the star HD 189733b. It is
```

图 2-1 基线测试图

Case 8/10: 当询问美国首任总统时，模型正确输出 "George Washington"，而非预期注入的 "Abraham Lincoln"。

Case 9/10: 询问鳄梨酱的主要成分，模型输出 "the avocado"，并未识别到预期的 "Tomato"。

Case 10/10: 询问离太阳最近的行星，模型根据原始天文知识输出 "Mercury"，而非目标 "Venus"。

结论：基线测试证明了原始模型在目标知识点上处于“旧知识”状态，为后续的编辑实验提供了清晰的对比基准。

3 Task 2: ROME 单条编辑实践

ROME 算法通过对前馈神经网络（FFN）特定层进行秩为 1 的权重更新，实现对特定事实的精准改写。

算法配置与实现

模型：GPT-2 XL (1.5B)。

编辑层数：根据 `gpt2-xl.yaml` 配置，主要作用于第 17 层（`transformer.h.17.mlp.c_proj.weight`）。

持久化策略：在 `edit_rome.py` 中，通过设置 `keep_original_weight=False`，将计算得到的 ΔW 永久作用于模型并保存至 `./edited_gpt2_rome`。如图 3-1 所示。

```
(my_env) root@autodl-container-654f118e52-2f9b91b5:~/autodl-tmp/my-editing-code# python edit_rome.py
Right vector shape: torch.Size([1600])
Deltas successfully computed for ['transformer.h.17.mlp.c_proj.weight']
New weights successfully inserted into ['transformer.h.17.mlp.c_proj.weight']
2026-05-10 17:08:07,898 - easyeditor.editors.editor - INFO - Execution 9 editing took 1.7321977615356445
05/10/2026 17:08:07 - INFO - easyeditor.editors.editor - INFO - Execution 9 editing took 1.7321977615356445
2026-05-10 17:08:07,946 - easyeditor.editors.editor - INFO - Evaluation took 0.04742789268493652
05/10/2026 17:08:07 - INFO - easyeditor.editors.editor - INFO - Evaluation took 0.04742789268493652
2026-05-10 17:08:07,946 - easyeditor.editors.editor - INFO - 9 editing: The planet closest to the Sun is -> Venus
{'pre': {'rewrite_acc': [0.0], 'portability': {}}, 'case_id': 9, 'requested_rewrite': {'prompt': 'The planet closest to the Sun is', 'target_new': 'Venus', 'Sun'}, 'time': 1.7321977615356445, 'post': {'rewrite_acc': [1.0], 'locality': {}, 'portability': {}}}
05/10/2026 17:08:07 - INFO - easyeditor.editors.editor - 9 editing: The planet closest to the Sun is -> Venus
{'pre': {'rewrite_acc': [0.0], 'portability': {}}, 'case_id': 9, 'requested_rewrite': {'prompt': 'The planet closest to the Sun is', 'target_new': 'Venus', 'Sun'}, 'time': 1.7321977615356445, 'post': {'rewrite_acc': [1.0], 'locality': {}, 'portability': {}}}

正在保存编辑后的模型至: ./edited_gpt2_rome...
模型保存成功! 现在你可以运行 evaluate.py 进行评估了。
```

图 3-1 ROME 单条编辑图

编辑效果对比

以“离太阳最近的行星”为例：

编辑前：输出为 "Mercury"。

编辑中：终端显示 Deltas successfully computed，单条编辑耗时约 **1.73 秒**。

编辑后：验证结果显示 The planet closest to the Sun is -> Venus，编辑成功率（`rewrite_acc`）提升至 **1.0 (100%)**。

4 Task 3: MEMIT 批量编辑分析

MEMIT 算法扩展了 ROME 的思路，支持通过闭式解（Closed-form solution）一次性向多个层注入成百上千条知识。

批量编辑数据 (500 条)

耗时情况：总计完成 500 条事实注入，耗时 **2989.31 秒**（约 49.8 分钟）。

资源消耗：显存峰值占用为 **8214.80 MB**。在 11GB 的 2080 Ti 上运行平稳，未出现 OOM，如图 4-1 所示。

```
05/10/2026 16:58:26 - INFO - easyeditor.editors.editor - 499 editing: Porto is a twin city of ->
{'pre': {'rewrite_acc': [0.0], 'portability': {}}, 'case_id': 499, 'requested_rewrite': {'prompt':
o'}, 'time': 5.073363304138184, 'post': {'rewrite_acc': [0.0], 'locality': {}, 'portability': {}}}

[任务完成] 批量编辑耗时: 2989.31 秒
[资源消耗] 批量编辑峰值显存占用: 8214.80 MB
```

图 4-1 MEMIT 批量编辑分析图

5 Task 4: 综合评估

实验通过 `evaluate.py` 加载 ROME 编辑后的模型，对 10 个测试样本进行了三维指标量化评估。如图 5-1 所示。

```
(my_env) root@autodl-container-654f118e52-2f9b91b5:~/autodl-tmp/my-editing-code# python evaluate.py
--- [Task 4] 正在加载编辑后的模型进行评估 ---
Loading checkpoint shards: 100%|

=====
主语          | ES (可靠性) | PS (泛化性) | NS (局部性)
-----
Twitter (X)   | 通过       | 失败       | 通过
France        | 通过       | 失败       | 失败
Harry Potter | 通过       | 通过       | 失败
Amazon        | 通过       | 通过       | 失败
world         | 通过       | 失败       | 失败
iPhone        | 通过       | 通过       | 失败
water         | 通过       | 通过       | 失败
United States | 通过       | 失败       | 失败
guacamole     | 通过       | 失败       | 失败
Sun           | 通过       | 通过       | 失败
=====
评估完成。如果 PS 和 NS 均为 '通过'，说明 ROME 编辑具有良好的局部性和泛化性。
```

图 5-1 指标量化评估图

指标汇总表

主语 (Subject)	ES (可靠性)	PS (泛化性)	NS (局部性)
Twitter (x)	通过	失败	通过
France	通过	失败	失败
Harry Potter	通过	通过	失败
Amazon	通过	通过	失败
world	通过	失败	失败
iPhone	通过	通过	失败
water	通过	通过	失败
United States	通过	失败	失败
guacamole	通过	失败	失败
Sun	通过	通过	失败
平均得分	100%	50%	10%

结果深度分析与失败总结

1. 可靠性 (ES) 表现完美：所有样本均能正确输出编辑后的目标答案。这说明 ROME 能精准地在特定 Prompt 下修改模型的 Logits 分布。
2. 泛化性 (PS) 瓶颈：仅 50% 的样本通过了同义改写测试。
失败原因：模型对新知识的理解仍高度依赖于特定的 Token 序列。这表明 GPT-2 XL 的语义对齐能力较弱，注入的知识未能完全解耦并整合进其更深层的语义空间。
3. 局部性 (NS) 崩塌：局部性测试的通过率仅为 10%。
 - 失败原因：这是本实验中最显著的问题。修改特定 FFN 权重虽然注入了新知识，但由于 GPT-2 XL 参数量较小（1.5B），权重空间极度紧凑。即使是秩为 1 的更新（Rank-One Update），也对模型原有的无关事实造成了“误伤”，导致局部性评估失效。这反映了在小规模模型上进行强力干预时的“附带损伤”风险。

6 总结：关于知识编辑对模型机制的思考

通过本次实验，我深刻理解了大模型知识存储的“定位-改写”机制。ROME/MEMIT 证明了模型可以通过修改极小比例的参数来完成知识更新，这种“参数化存储”的灵活性令人惊叹。

然而，实验结果也揭示了其局限性：知识编辑并非万能。特别是局部性保护与语义泛化之间的权衡，是未来该领域的核心痛点。在实际应用中，对于 GPT-2

这一级别的模型，可能需要更精细的层选择策略或结合非参数化方法（如 RAG）来弥补知识冲突带来的副作用。